

Comandos de Repetição em Python

Comandos de repetição permitem executar um bloco de código várias vezes. Eles são úteis para automatizar tarefas repetitivas e iterar sobre coleções de dados. Em Python, os principais comandos de repetição são `for` e `while`.

1. Loop `for`

O loop `for` é usado para iterar sobre uma sequência (como uma lista, tupla ou string) ou outro objeto iterável.

Exemplo:

```
frutas = ["maçã", "banana", "cereja"]

for fruta in frutas:
    print(f"Eu gosto de {fruta}")
```

2. Iteração sobre um Intervalo de Números com `range`

A função `range` é usada para gerar uma sequência de números, que pode ser iterada com um loop `for`.

Exemplo:

```
for numero in range(1, 6):
    print(f"Número: {numero}")
```

3. Loops `for` Aninhados

Loops `for` podem ser aninhados dentro de outros loops `for` para iterar sobre múltiplas sequências simultaneamente.

Exemplo:

```
lista1 = [0, 1, 2, 3, 4]
lista2 = [1, 2, 3]

for elemento_lista1 in lista1:
    for elemento_lista2 in lista2:
        print(elemento_lista1 * elemento_lista2)
    print('----')
```

4. Uso de `break` e `continue` em Loops `for`

`break`

O comando `break` é usado para interromper um loop antes que ele tenha percorrido todos os itens.

Exemplo:

```
for numero in range(1, 10):
    if numero == 5:
        break
    print(f"Número: {numero}")
```

continue

O comando `continue` é usado para pular a iteração atual e continuar com a próxima.

Exemplo:

```
for numero in range(1, 10):
    if numero == 5:
        continue
    print(f"Número: {numero}")
```

5. Iteração com `enumerate`

O comando `enumerate` permite iterar sobre um objeto iterável e obter um contador para cada elemento.

Exemplo:

```
animais = ["cachorro", "gato", "elefante"]

for indice, animal in enumerate(animais):
    print(f"{indice}: {animal}")
```

6. Loop `while`

O loop `while` continua executando um bloco de código enquanto uma condição especificada é verdadeira.

Exemplo:

```
contador = 1

while contador <= 5:
    print(f"Contador: {contador}")
    contador += 1
```

7. Uso de `break` e `continue` em Loops `while`

break

O comando `break` pode ser usado em um loop `while` para interromper a execução do loop.

Exemplo:

```
contador = 1

while contador <= 10:
    if contador == 5:
        break
    print(f"Contador: {contador}")
    contador += 1
```

continue

O comando `continue` pode ser usado em um loop `while` para pular a iteração atual e continuar com a próxima.

Exemplo:

```
contador = 1

while contador <= 10:
    contador += 1
    if contador == 5:
        continue
    print(f"Contador: {contador}")
```

8. Controle de Loops com Condições e Contadores

É comum usar variáveis de controle e contadores para gerenciar a execução de loops.

Exemplo:

```
n = 5
soma = 0
contador = 1

while contador <= n:
    soma += contador
    contador += 1

print(f"Soma dos primeiros {n} números: {soma}")
```

9. Exemplos Adicionais

Exemplo 1: Tabuada

```
for i in range(1, 11):
    for j in range(1, 11):
        print(f"{i} x {j} = {i * j}")
    print("-----")
```

Exemplo 2: Números Primos

```
for num in range(2, 20):
    primo = True
    for i in range(2, num):
```

```
    if num % i == 0:
        primo = False
        break
if primo:
    print(f"{num} é um número primo")
```

Exemplo 3: Fibonacci

```
a, b = 0, 1
while b < 100:
    print(b, end=", ")
    a, b = b, a + b
```

10. Referências

- [Documentação Oficial do Python](#)
- [Guia de Estilo para Código Python \(PEP 8\)](#)